The Structure of Tail-Biting Trellises: Minimality and Basic Principles

Ralf Koetter, *Member, IEEE,* and Alexander Vardy, *Fellow, IEEE*

***Abstract*—Basic structural properties of tail-biting trellises are investigated. We start with rigorous definitions of various types of minimality for tail-biting trellises. We then show that biproper and/or nonmergeable tail-biting trellises are not necessarily minimal, even though every minimal tail-biting trellis is biproper. Next, we introduce the notion of linear (or group) trellises and prove, by example, that a minimal tail-biting trellis for a binary linear code need not have any linearity properties whatsoever. We observe that a trellis—either tail-biting or conventional—is linear if and only if it factors into a product of elementary trellises. Using this result, we show how to construct, for any given linear code $\mathbb{C}$, a tail-biting trellis that minimizes the *product* of state-space sizes among all possible linear tail-biting trellises. We also prove that *every* minimal linear tail-biting trellis for $\mathbb{C}$ arises from a certain $n \times n$ *characteristic matrix*, and show how to compute this matrix in time $O(n^2)$ from any basis for $\mathbb{C}$. Furthermore, we devise a linear-programming algorithm that starts with the characteristic matrix and produces a linear tail-biting trellis for $\mathbb{C}$ which minimizes the *maximum* state-space size. Finally, we consider a generalized product construction for tail-biting trellises, and use it to prove that a linear code $\mathbb{C}$ and its dual $\mathbb{C}^{\perp}$ have the same state-complexity profiles.**

***Index Terms*—Block codes, characteristic matrix, codes on graphs, convolutional codes, linearity, minimal trellises, tail-biting trellises, trellis-complexity.**

I. Introduction

TRELLIS representations of linear block codes have received much attention in recent years [3], [14], [16], [17], [21], [31], [35]. Such representations not only illuminate code structure, but also often lead to efficient trellis-based decoding algorithms. It is now well known that, given a specific coordinate ordering, there exists a unique, up to isomorphism, minimal (conventional) trellis for any linear block code. The minimal trellis for a linear code $\mathbb{C}$ simultaneously minimizes all the conceivable measures of trellis complexity, and can be easily constructed from a generator matrix or a parity-check matrix for $\mathbb{C}$.

On the other hand, much less is known about *tail-biting* trellises. Tail-biting trellis representations are interesting for several reasons. First, the complexity of a tail-biting trellis may be much lower than the complexity of the best possible conventional trellis. It is shown in [3], [35] that the number of states in a tail-biting trellis for a linear code $\mathbb{C}$ can be as low as the *square root* of the number of states in the minimal conventional trellis for $\mathbb{C}$. Second, tail-biting trellises may be considered as the simplest form of a *factor graph* with cycles. The recent development of iterative decoding techniques [35], [15] has led to a vivid interest in factor graphs. While the performance of iterative decoding on general graphs with cycles remains somewhat of a mystery, iterative decoding of tail-biting trellises is by now reasonably well understood [1], [7], [23], [26].

Thus, the major remaining problem with tail-biting trellises is that of efficient construction. Given a linear block code $\mathbb{C}$ over the finite field $\mathbb{F}_q$, how can one construct a minimal tail-biting trellis for $\mathbb{C}$? Although several *examples* of such trellises are known [3], the understanding of minimal tail-biting trellises is not on par with conventional trellises. Our goal in this work is to lay out some of the foundations of the theory of tail-biting trellises.

We start with a definition of both conventional and tail-biting trellises in the next section. We then define *reduced* and *one-to-one* tail-biting trellises. In addition to the usual labeling of edges in a trellis, we will also need to deal with labelings of *vertices*. Thus, we introduce the concepts of a *labeled trellis* and the corresponding *label code*. We further define a *representation code* of a labeled trellis T as a subset of the label code of T, from which the entire trellis can be uniquely reconstructed. This definition leads to the notion of a *representation matrix* for a (linear) trellis, which is introduced in Section IV.

In Section III, we present rigorous definitions of various types of minimality for tail-biting trellises. The fact that the minimal conventional trellis for a linear code is unique makes it possible to define minimality in a number of equivalent ways: any reasonable definition leads to the same unique minimal trellis (cf. [31], [33]). The situation is considerably more involved for tail-biting trellises. Minimal tail-biting trellises are usually not unique, and special care needs to be taken to define minimality itself. We thus distinguish between different types of minimality that correspond to different (partial) orders on the set of trellises for a given code. We also show that every minimal tail-biting trellis is biproper; on the other hand, biproper and/or nonmergeable tail-biting trellises are not necessarily minimal. It follows that iterative merging of vertices in a nonminimal trellis leads to minimality for conventional trellises but not for tail-biting trellises.

In Section IV, we introduce the notion of *linear* (or *group*) trellises. Loosely speaking, linearity means that the set of edge/vertex label sequences is closed under componentwise operation in the appropriate algebraic domain—a field or a group.

Manuscript received June 5, 2002; revised May 11, 2003. This work was supported by the David and Lucile Packard Foundation and by the National Science Foundation. The material in this paper was presented in part at the IEEE Information Theory Workshop, Killarney, Ireland, July 1998 and in part at the IEEE International Symposium on Information Theory, Lausanne, Switzerland, July 2002.

R. Koetter is with the Coordinated Science Laboratory, University of Illinois at Urbana-Champaign, Urbana, IL 61801 USA (e-mail: koetter@csl.uiuc.edu).

A. Vardy is with the Department of Electrical and Computer Engineering, the Department of Computer Science, and the Center for Wireless Communications, University of California, San Diego, La Jolla, CA 92093 USA (e-mail: vardy@kilimanjaro. ucsd.edu).

Communicated by S. Litsyn, Associate Editor for Coding Theory.

Digital Object Identifier 10.1109/TIT.2003.815769

In [11], we show how to decide whether a given (unlabeled) trellis is linear: we provide an algorithm that constructs an appropriate labeling of the vertices if the trellis is linear, and halts if it is not. An interesting outcome of this investigation is that trellis linearity is inherently a graph-theoretic property: we give an example of two trellises over the trivial alphabet $\mathcal{A} = \{0\}$ one of which is linear while the other is not. Another surprising observation is that a minimal tail-biting trellis for a linear code may be nonlinear: we construct a minimal trellis for a simple linear code over $\mathbb{F}_2$ that has no linearity properties[1] whatsoever.

We then discuss the relationship between trellis linearity and the *product construction*. The product construction for conventional trellises was introduced by Kschischang and Sorokine in [16]. It was later employed in the context of tail-biting trellises by Calderbank, Forney, and Vardy [3]. One of our key results in [11] is that a trellis—either tail-biting or conventional—is linear if and only if it may be obtained by a product construction. Thus, every linear trellis factors into a product of elementary trellises over a field, and every abelian-group trellis factors into elementary trellises over a group.

It follows that the search for a minimal linear trellis reduces to the task of specifying the appropriate input to the product construction: given a linear code $\mathbb{C}$, we need to specify a set of *generators* for $\mathbb{C}$ and then specify a *span* for each generator. In contrast to the case of conventional trellises, when constructing a tail-biting trellis, one can *freely choose* the span of each generator, and there are at least $O(n^k)$ ways to do so for a code of length n and dimension k. Nevertheless, in Section V, we present a general solution to the problem of constructing minimal linear tail-biting trellises for linear codes. To this end, we introduce the notion of a *characteristic matrix* for a linear code $\mathbb{C}$. This is an $n \times n$ matrix that can be easily computed in time $O(n^2)$ from any basis for $\mathbb{C}$. We show that although minimal tail-biting trellises are generally not unique, *every* minimal linear tail-biting trellis for $\mathbb{C}$ necessarily arises from the characteristic matrix for $\mathbb{C}$. We also investigate some of the properties of the characteristic matrix, and of the associated *span matrix*. This investigation enables us to show, in particular, that certain tail-biting trellises for a linear code $\mathbb{C}$ and for its dual $\mathbb{C}^\perp$ have the same state-complexity profile.

In Section VI, we start with the characteristic matrix for a linear code $\mathbb{C}$, and show how to construct a linear tail-biting trellis for $\mathbb{C}$ that minimizes the *product* of the vertex-space sizes among all possible linear trellises for $\mathbb{C}$ (a $\prec_\Pi$-minimal trellis). We also propose a linear-programming algorithm that produces a linear tail-biting trellis for $\mathbb{C}$ which minimizes the *maximum* vertex-space size (a $\prec_{\max}$-minimal trellis). As is common in the case of integer-programming problems, we do not have a precise estimate of the running time of the proposed algorithm. Nevertheless, we found that it works extremely well in practice.

Finally, in Section VII, we discuss a generalization of the product construction for tail-biting trellises. The idea is to replace the *sum* of the edge labels in the original Kschischang–Sorokine construction [16] with an *arbitrary function*. The generalized product construction enables us to produce a tail-biting trellis for a linear code $\mathbb{C}$ directly from a parity-check matrix for $\mathbb{C}$, rather than from a generator matrix. This leads to the following duality result: given any linear trellis T for $\mathbb{C}$, there exists a *dual trellis* $T^\perp$ that represents the dual code $\mathbb{C}^\perp$, such that the state-complexity profile of $T^\perp$ is componentwise less than or equal to that of T. Forney [6] derived a similar result using quite different tools, in the context of normal (generalized) state realizations of codes on graphs.

We note that all of the results in this paper deal with trellis representation of *linear codes*, given a *fixed order* of the time axis. In the case of conventional trellises, comprehensive theory has been developed on this subject (cf. [31]), and it is fair to say that minimal conventional trellises for linear codes are by now very well understood. On the other hand, the problem of finding the best *permutation* of the time axis for a given linear code is still open, even in the case of conventional trellises. The situation is different for tail-biting trellises in that very little was known regarding both problems—the permutation problem as well as the minimality problem for a fixed time axis. While our results here do not answer all the questions that pertain to minimal tail-biting trellises for a fixed time axis, we do provide solutions to some of the main problems in this domain. We thus hope to bring the theory of tail-biting trellises on par with that of conventional trellises.

II. Preliminaries

We start with the definitions of conventional and tail-biting trellises. We also introduce a number of concepts related to tail-biting trellises, that will be used throughout the paper.

An edge-labeled directed *graph* is a triple $(V, E, \mathcal{A})$, consisting of a set V of *vertices*, a finite set $\mathcal{A}$ called the *alphabet*, and a set E of ordered triples (v, a, v'), with $v, v' \in V$ and $a \in \mathcal{A}$ called *edges*. We say that an edge $(v, a, v') \in E$ begins at v, ends at v', and has label a.

Definition 2.1: A conventional trellis $T = (V, E, \mathcal{A})$ of depth n is an edge-labeled directed graph with the following property: the vertex set V can be partitioned as

$$V = V_0 \cup V_1 \cup \cdots \cup V_n \tag{1}$$

such that every edge in T begins at a vertex of V_i and ends at a vertex of V_{i+1}, for some $i = 0, 1, \ldots, n-1$. The sets $V_0, V_1, \ldots, V_n$ are called the *vertex classes* of T. The ordered index set $\mathcal{I} = \{0, 1, \ldots, n\}$ induced by the partition in (1) is called the *time axis* for T.

If every vertex in T lies on at least one path from a vertex in V_0 to a vertex in V_n, we say that T is *reduced*. The sequence of edge labels along each path of length n in T defines an ordered n-tuple over the label alphabet $\mathcal{A}$. We say that T *represents* a block code $\mathbb{C}$ of length n over $\mathcal{A}$, if $\mathbb{C}$ is precisely the set of all such n-tuples.

Tail-biting trellises have been traditionally used (cf. [19]) as a means of terminating a convolutional code without incurring a rate loss. More recently, tail-biting trellises for *block codes* were considered in [3], [8], [10], [18], [26], [25], [27], [28],

[1]Examples given in [3] show that a minimal tail-biting trellis for a linear code over $\mathbb{F}_q$ may be a group trellis over the additive group of $\mathbb{F}_q$. Such trellises are still essentially linear—they factor into a product of elementary trellises over a group, as described in Section IV. Our example is fundamentally different in that the minimal trellis we construct is not a group trellis and cannot be factored.